

Optimized Deep Learning - Jean Zay

Training and large batches



INSTITUT DU
DÉVELOPPEMENT ET DES
RESSOURCES EN
INFORMATIQUE
SCIENTIFIQUE

cc BY NC SA

Loss Landscape

Loss Landscape ◂

Residual Learning ◂

Initialization ◂

Loss Landscape

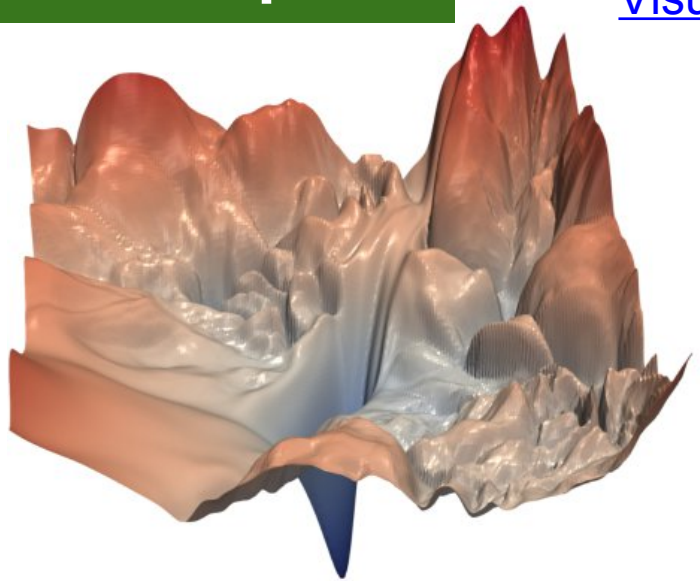
https://losslandscape.com/



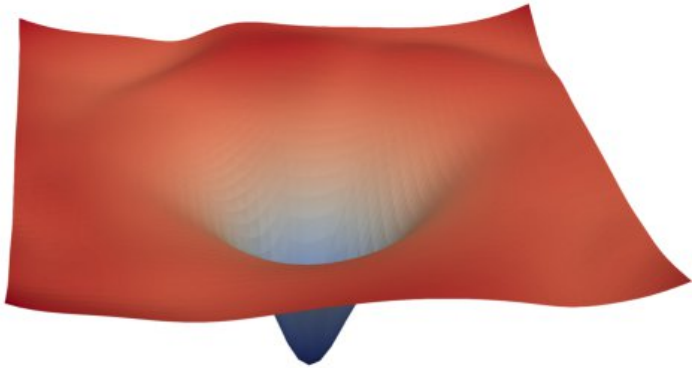
Loss Landscape

[Visualizing the Loss Landscape of Neural Nets]

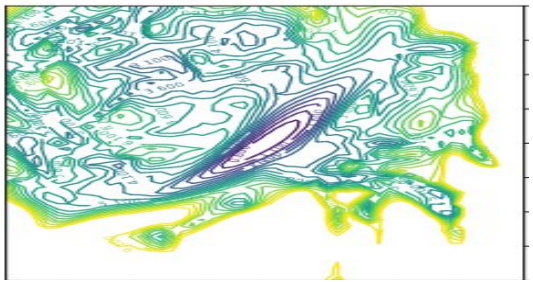
Residual Learning
Since Resnets (2015) ...

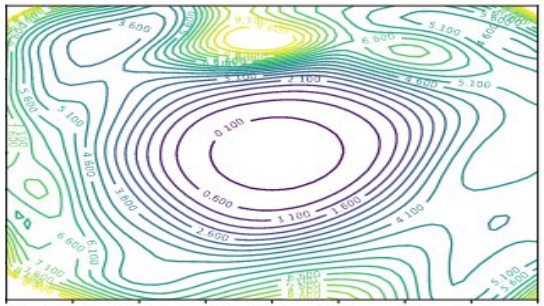


(a) without skip connections

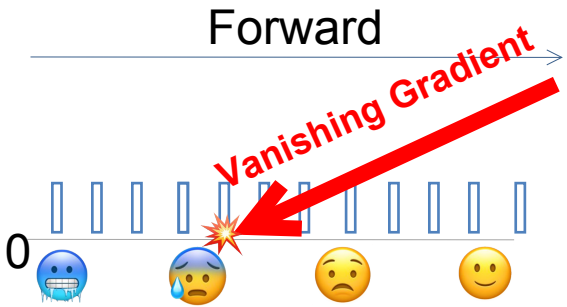


(b) with skip connections



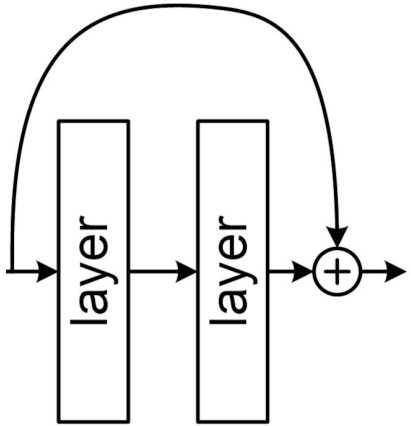


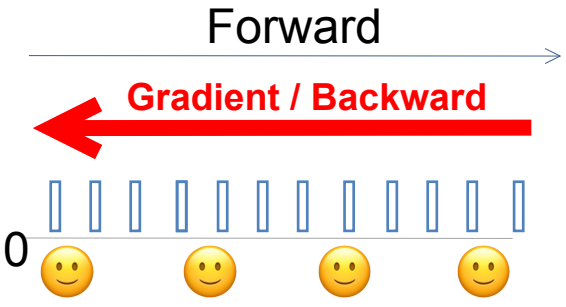
Residual Learning



Vanishing Gradient issue
without skip connections

Residual Block

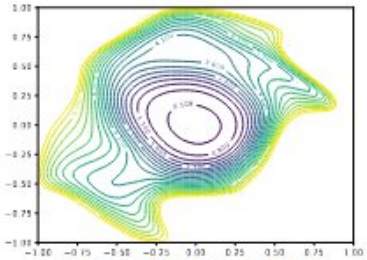




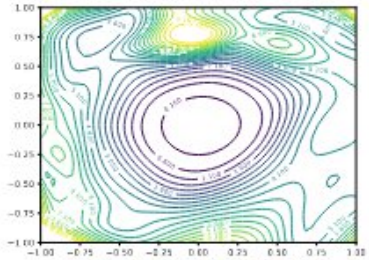
No Vanishing Gradient issue
with skip connections

Residual Learning – depth impact

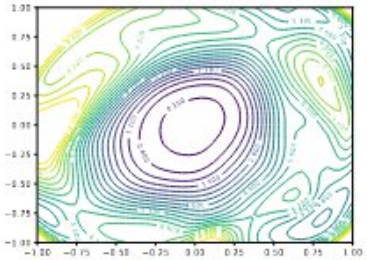
ResNet-20



(a) ResNet-20, 7.37%

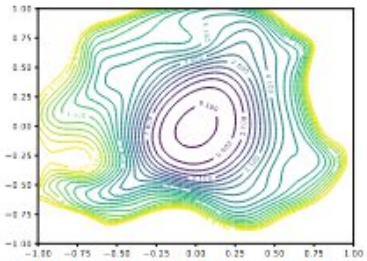


(b) ResNet-56, 5.89%

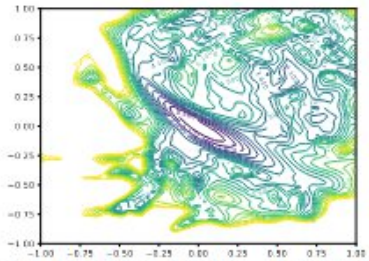


(c) ResNet-110, 5.79%

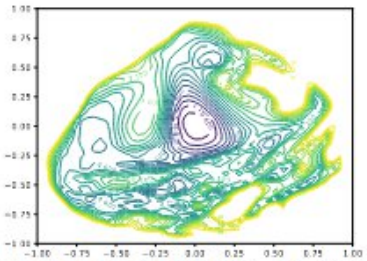
ResNet-20
without skip connections



(d) ResNet-20-NS, 8.18%



(e) ResNet-56-NS, 13.31%

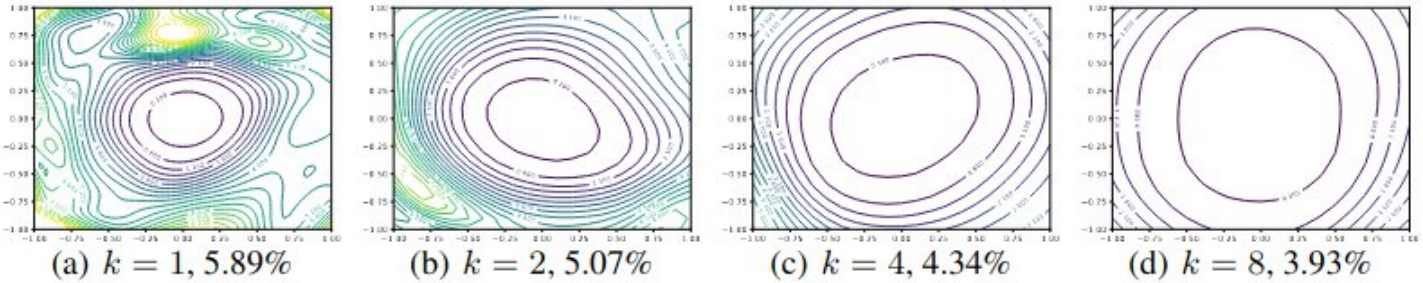


(f) ResNet-110-NS, 16.44%

Skip connections prevent increasing depth from making the loss landscape excessively sharp and chaotic

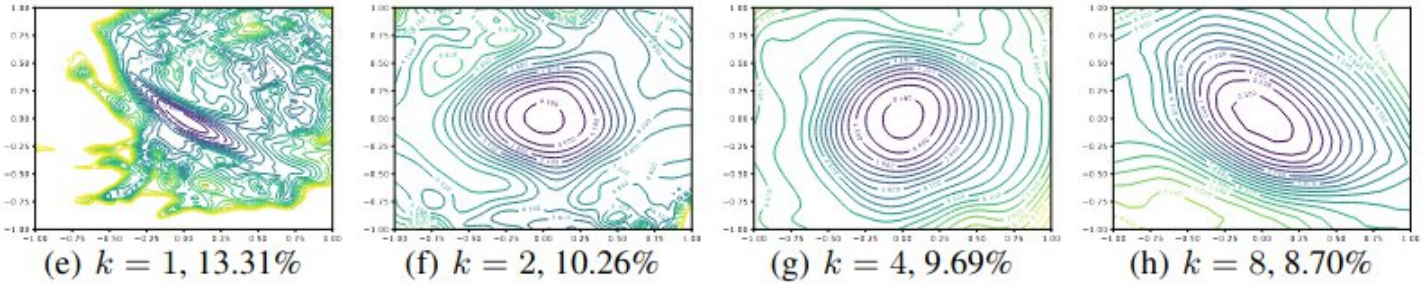
Residual Learning – width impact

Wide-ResNet-56



(a) $k = 1$, 5.89% (b) $k = 2$, 5.07% (c) $k = 4$, 4.34% (d) $k = 8$, 3.93%

Wide-ResNet-56
without skip connections



(e) $k = 1$, 13.31% (f) $k = 2$, 10.26% (g) $k = 4$, 9.69% (h) $k = 8$, 8.70%

Skip connections prevent narrow networks from developing highly irregular loss landscapes

k = filter coefficient compared to ResNet-56

Model Parameters Initialization

The Blessing of Dimensionality :



By default in PyTorch:

- Best initialization algorithm depending on the type of layer (linear, convolutional, transform, ...).
- Today, it is no longer necessary to try to optimize initialization.

Learning Rate Scheduler

Learning Rate scheduler ◂

Cyclic scheduler ◂

One Cycle scheduler ◂

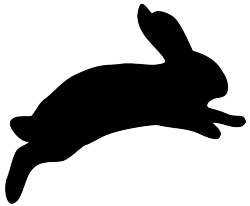
ScheduleFree ◂

LR Finder◂

Learning Rate

$$\underbrace{\theta}_{\text{Gradient}} = \theta - \underbrace{\eta}_{\text{LR}} \underbrace{\frac{\partial J(\theta)}{\partial \theta}}_{\text{Gradient}}$$

High Learning-rate



+ Larger steps (faster convergence)
+ Escape local minima
+ Good for early steps
- Risk skipping acceptable minima
- Risk “bouncing” over minima

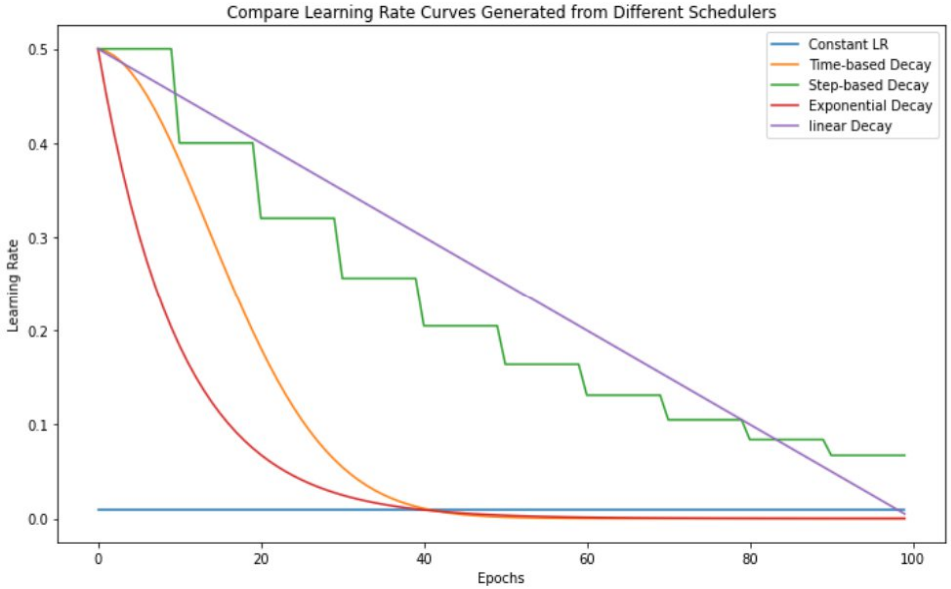
Small Learning-rate

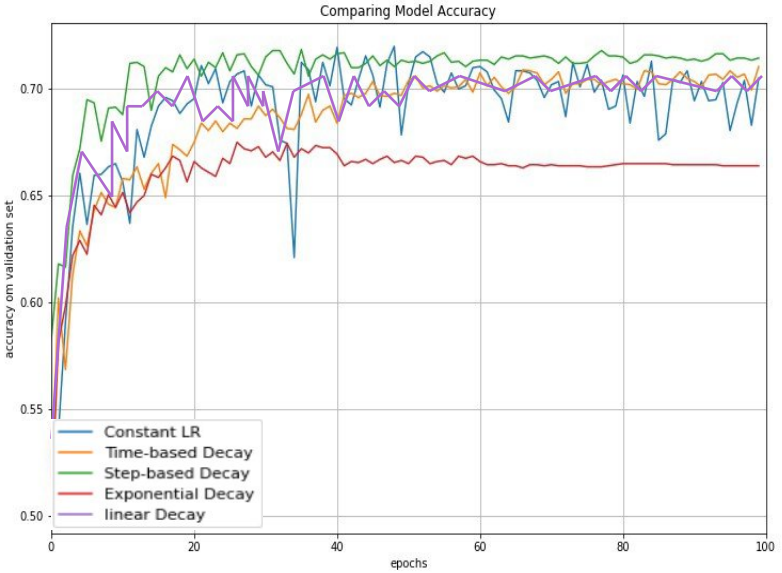


+ Don’t skip local minima
+ Converge towards bottom of minima
+ Good for later steps
- Smaller steps (slower convergence)
- Risk getting stuck in local minima

Learning Rate Scheduler

Learning rate decay

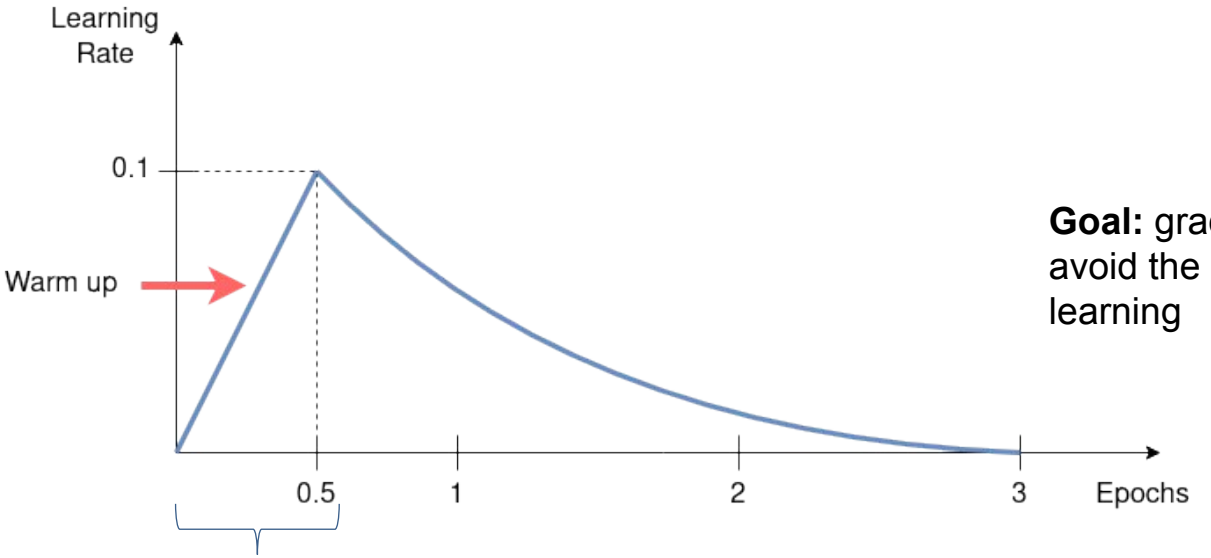




Learning Rate Scheduler

[WARMUP for large batches]

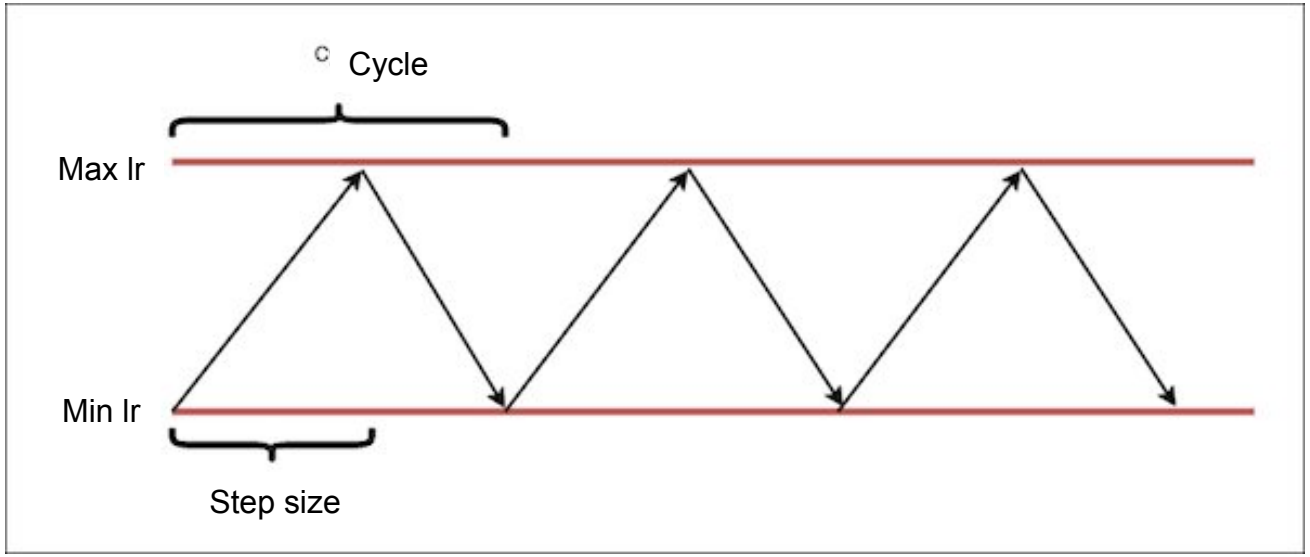
Problems: The first iterations have too much effect on the model (significant losses, high gradients, bias, etc.), a high learning rate can cause strong instability or divergence



Goal: gradually increase the learning rate to avoid the risk of divergence at the start of learning

Cyclic Learning Rate Scheduler

Cyclical Learning Rates for Training Neural Networks - *Leslie N. Smith* *2017*

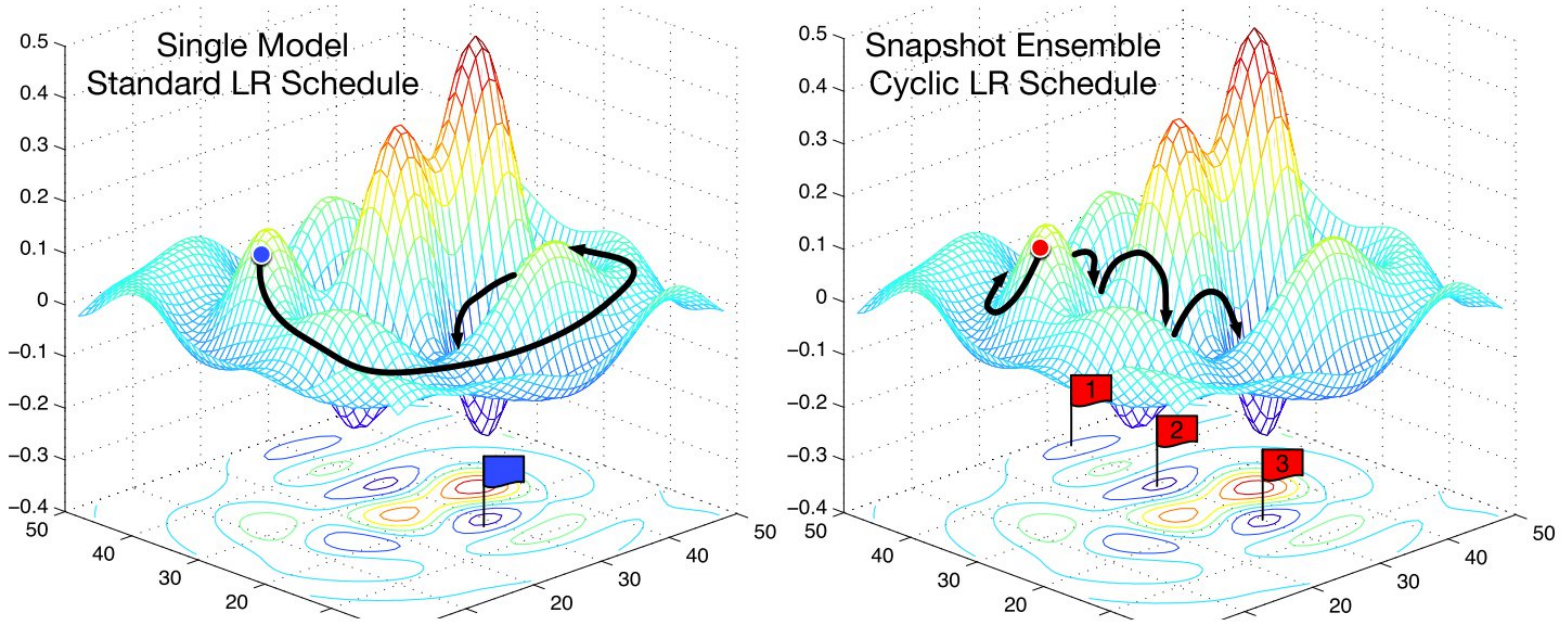


Paramètres :

- Step_size = x * epoch ($2 \leq x \leq 10$)
- Base_lr -> min convergence value
- max_lr -> max convergence value

Succession of warmups and learning rate decays

Cyclic Learning Rate Scheduler

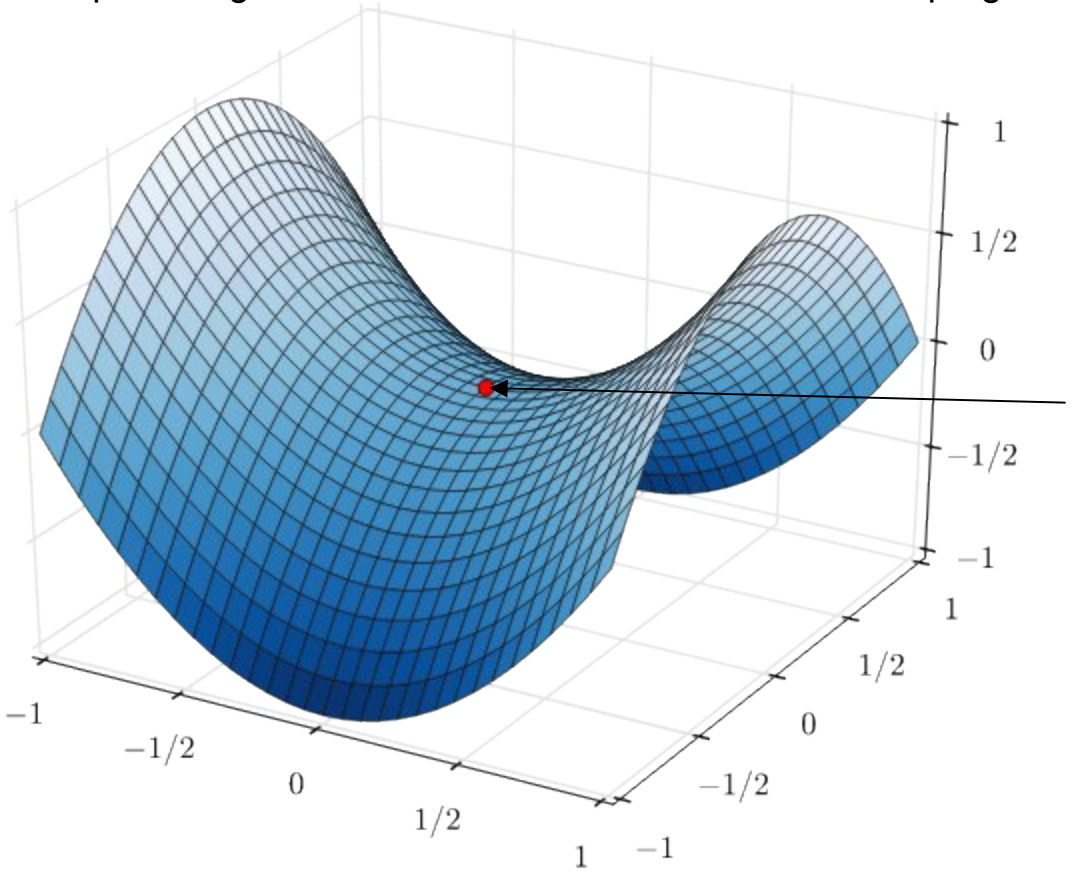


SNAPSHOT ENSEMBLES: TRAIN 1, GET M FOR FREE

Gao Huang, Yixuan Li, Geoff Pleiss

Saddle point Problem

Saddle point: A gradient close to zero which will make the progression of the model very slow

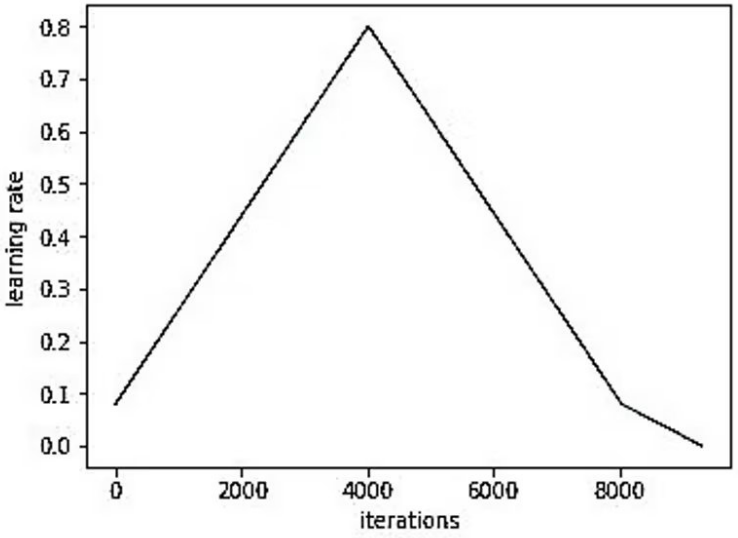


How do I leave the saddle point?

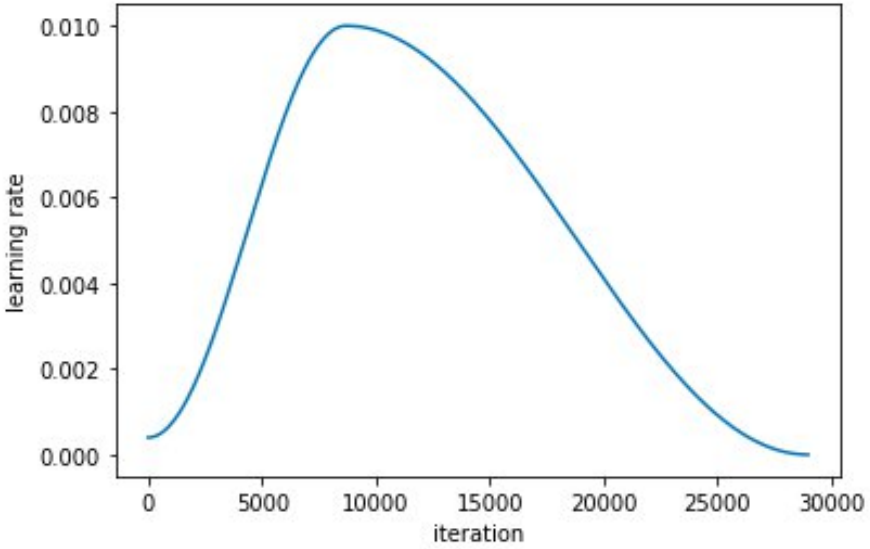
One Cycle Learning Rate

One cycle is enough! [A disciplined approach to neural network hyper-parameters](#) - *Leslie N. Smith* *2018*

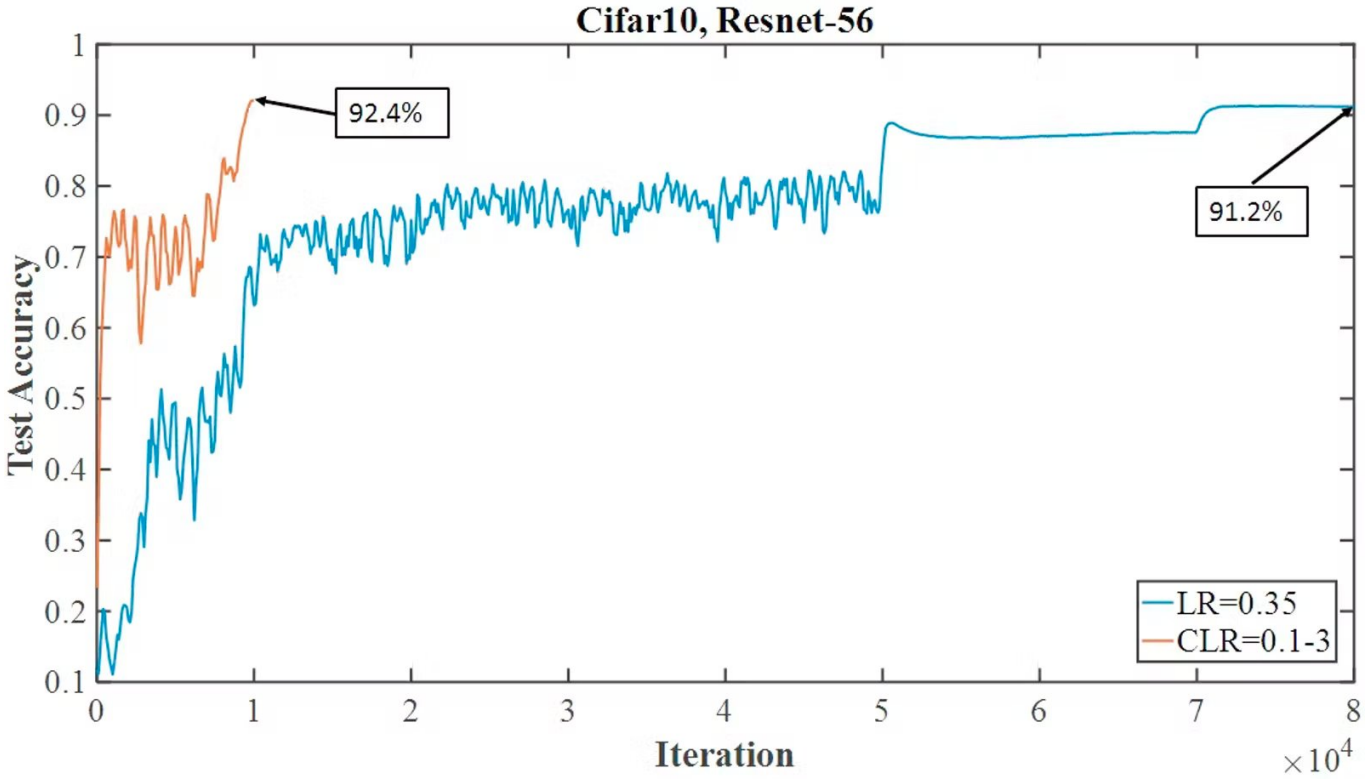
Proposition initiale



cosine annealing : Recommandation par FastAI



One Cycle Learning Rate - *Super convergence*



Faster convergence for equivalent final precision

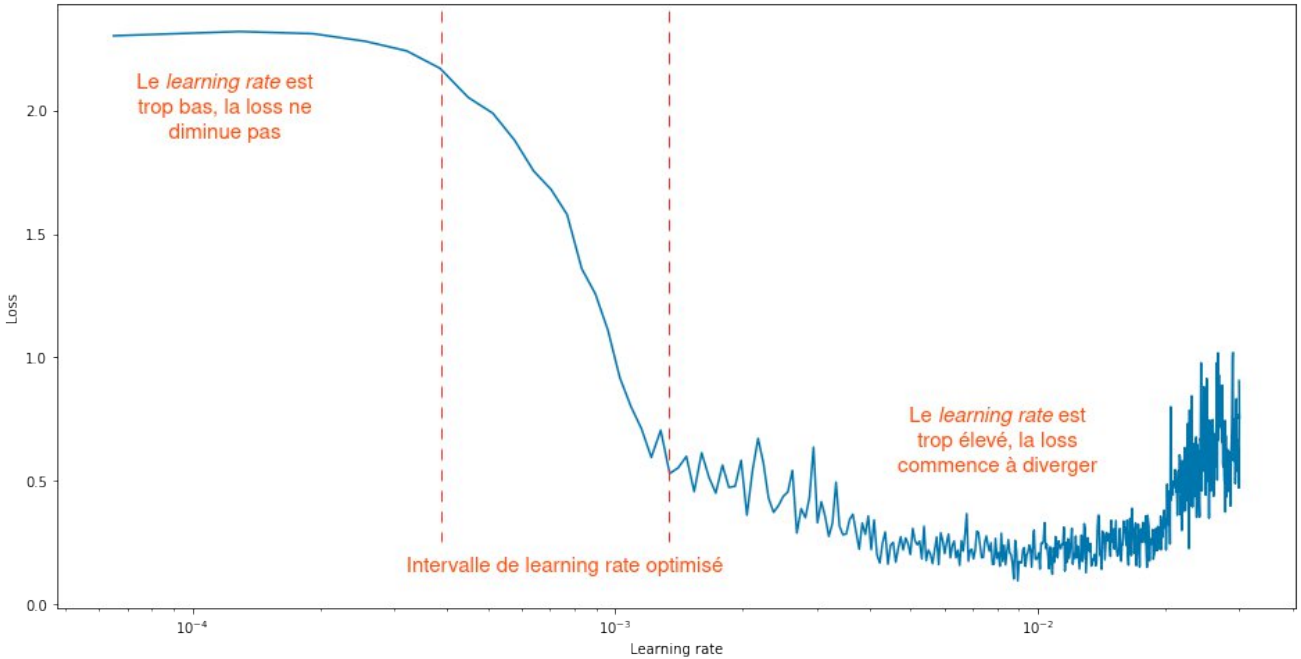
Learning Rate Finder

Goal: Find the **optimal learning rate** values for your model, particularly for **the maximum value** of a *cyclic scheduler*

Run your model over a few epochs by increasing its learning rate

- **Start of loss reduction**
- **→ Minimal learning rate**

- **Start of loss variation**
- **→ Maximum learning rate**



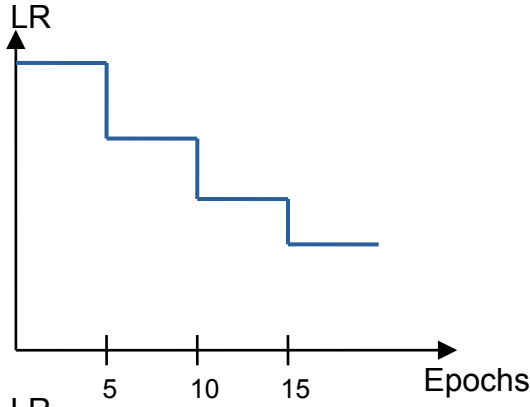
Learning Rate Scheduler

Each **scheduler** has *its own settings*

```
import torch.optim as opt

scheduler = opt.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

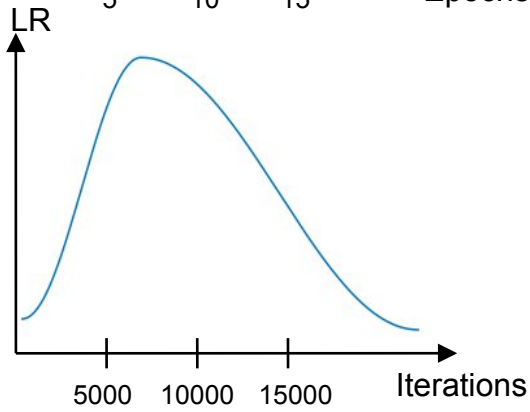
for epoch in range(100):
        train(...)
        validate(...)
        scheduler.step()
```




```
import torch.optim as opt

scheduler = opt.lr_scheduler.CyclicLR(optimizer, base_lr=0.01, max_lr=0.1)


for epoch in range(10):
        for batch in data_loader:
                train_batch(...)
                scheduler.step()
```



Gradient Descent Optimizer

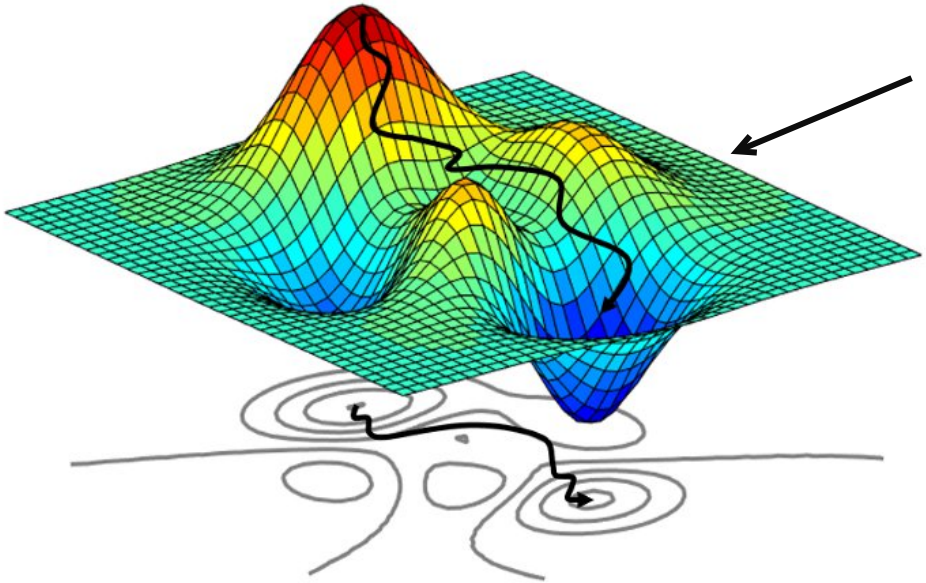
SGD ◂

ADAM◂

ADAMW ◂

Optimizer - SGD

The **optimizer** is the algorithm that **controls the gradient descent** and **the minimum search** with the aim of optimizing the learning time and the final metric.



SGD = Stochastic Gradient Descent
Calculating the Gradient and updating the weights at each batch

+ Batch size and learning rate adaptable according to conflicting needs:
- Exploration to find the best local minimum
- Acceleration of gradient descent

SGD *with Momentum*

Goal: Take **previous gradients** into consideration for **faster** gradient descent.

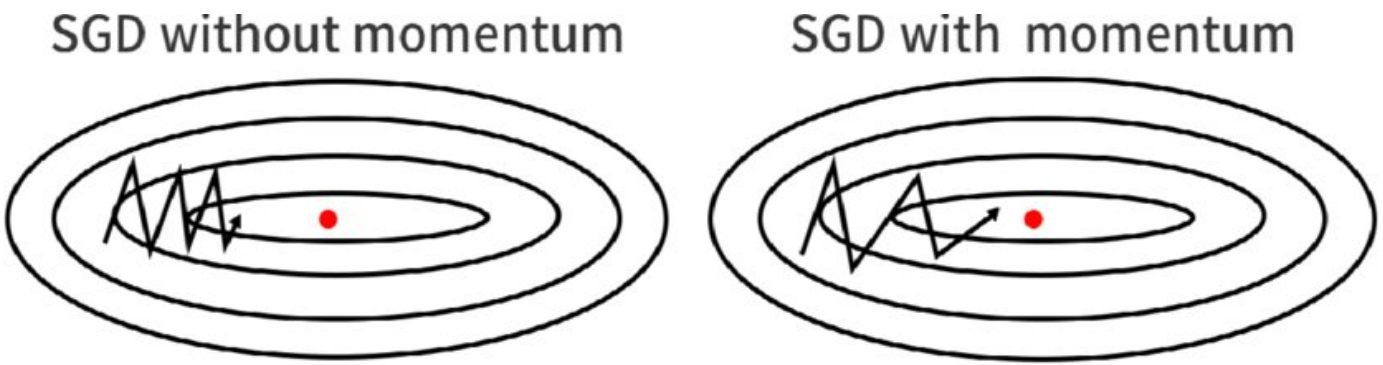
$$m_0 = 0$$

$$m_i = \boxed{\beta} * m_{i-1} + (1 - \beta) * g_i$$

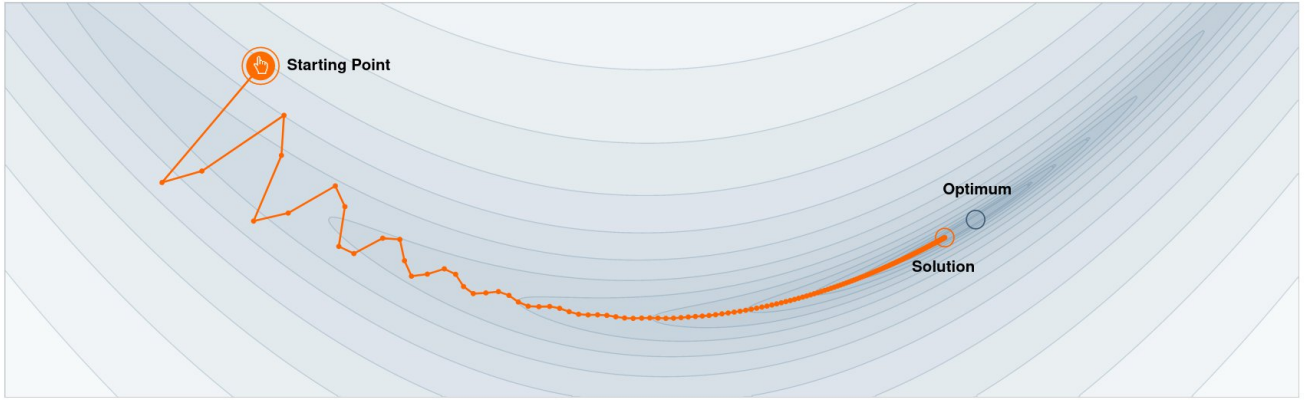
Momentum coefficient

$$\theta_i = \theta_{i-1} - \alpha * m_i$$

+ Allows you to **converge more quickly**
- **No guarantee** that momentum will take us in the right direction



Why Momentum Works ?



Step-size α = 0.02

0 0.003 0.006

Momentum β = 0.99

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

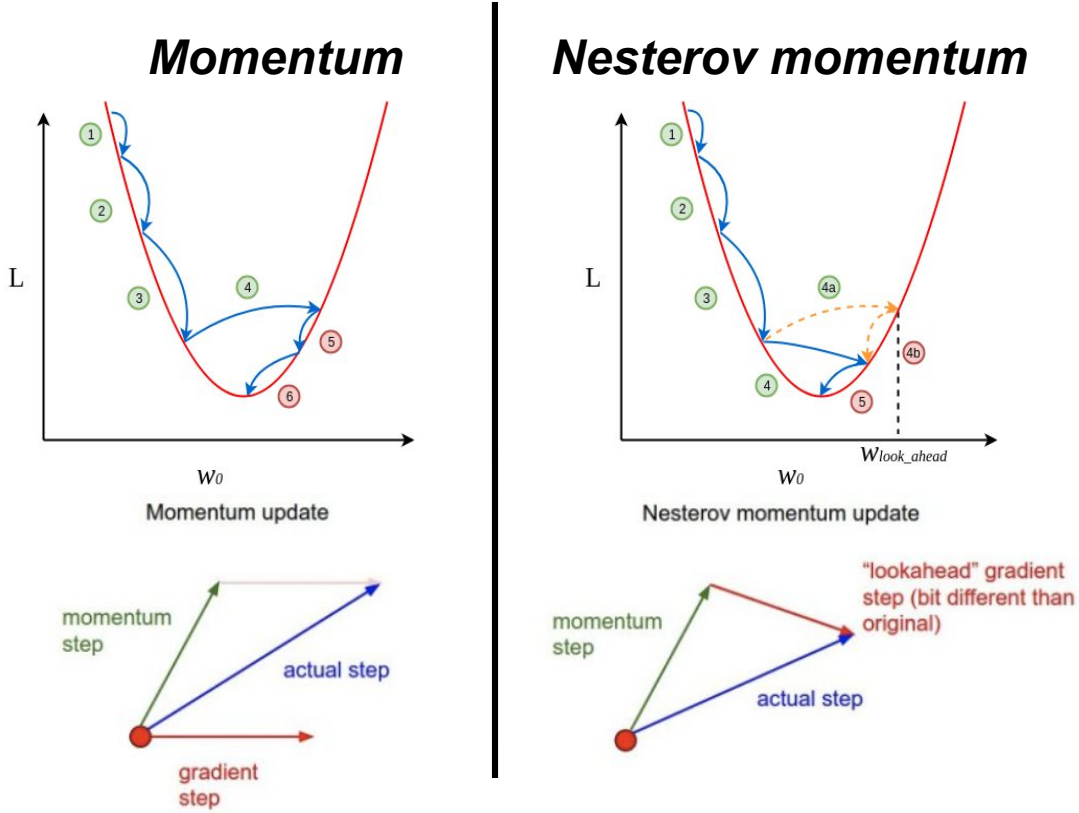
GABRIEL GOH
UC Davis

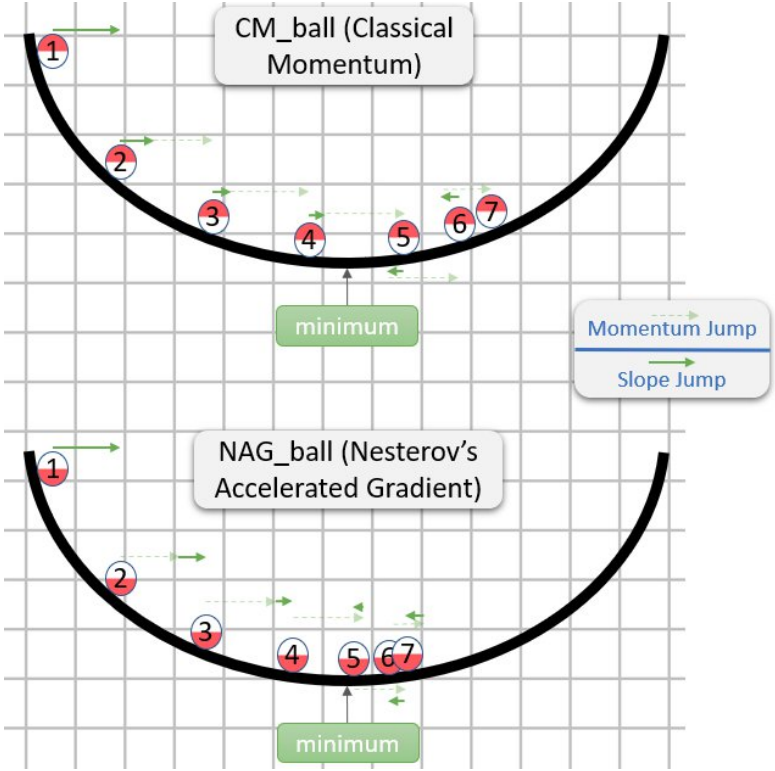
April. 4
2017

Citation:
Goh, 2017

https://distill.pub/2017/momentum/

Nesterov accelerated gradient





In Nesterov momentum, we first look ahead in the momentum direction, then compute the gradient at the anticipated position.

Root Mean Square Propagation

Each **gradient element is squared** to make large gradients easier to detect

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1-\beta)\mathbf{g}_t^2$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\alpha}{\sqrt{m_t} + \epsilon} g_t$$

Normalisation :

→ Large recent squared gradients ⇒ $\boldsymbol{m_t}$ is large ⇒ smaller effective learning rate

→ Small recent squared gradients ⇒ $\boldsymbol{m_t}$ is small ⇒ larger effective learning rate

RMSProp highlights large gradients,
without letting them produce excessively large weight updates

ADAptive Moment estimation

$$m_i = \beta_1 * m_{i-1} + (1 - \beta_1) * g_i$$

First moment : SGD like

$$v_i = \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2$$

Second moment : RMSPROP like

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}$$

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^i}$$

Normalisation : Prevents the optimizer from underestimating gradients at the beginning of training

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i$$

Normalisation : RMSPROP like

Adam modifies both the magnitude and the direction of the update for each weight.

MomentUm Orthogonalized by Newton-Schulz

Algorithm 2 Muon

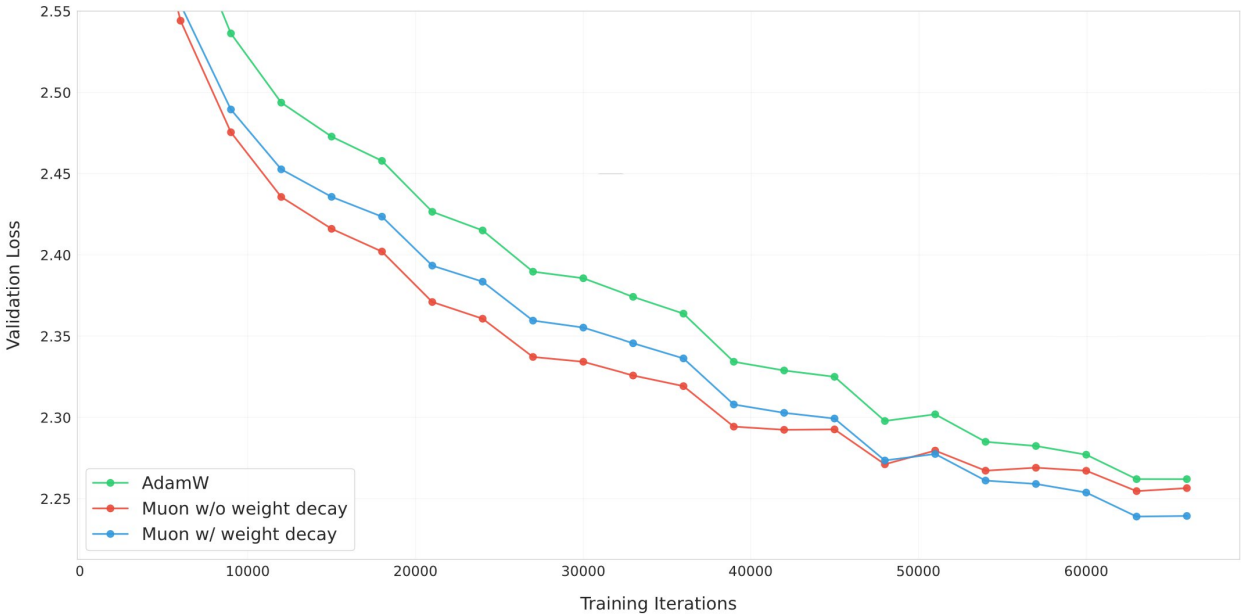
Require: Learning rate η, momentum μ

1: Initialize $B_0 \leftarrow 0$
2: **for** $t = 1, \ldots$ **do**
3: Compute gradient $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$
4: $B_t \leftarrow \mu B_{t-1} + G_t$ Adjust the gradient using momentum
5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$ Preserving its direction while normalizing its magnitude
6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
7: **end for**
8: **return** θ_t

Muon emphasizes the orientation of the momentum-adjusted gradient rather than its magnitude

MomentUm Orthogonalized by Newton-Schulz

Muon trains Transformers faster than Adam but is less well suited to CNN architectures

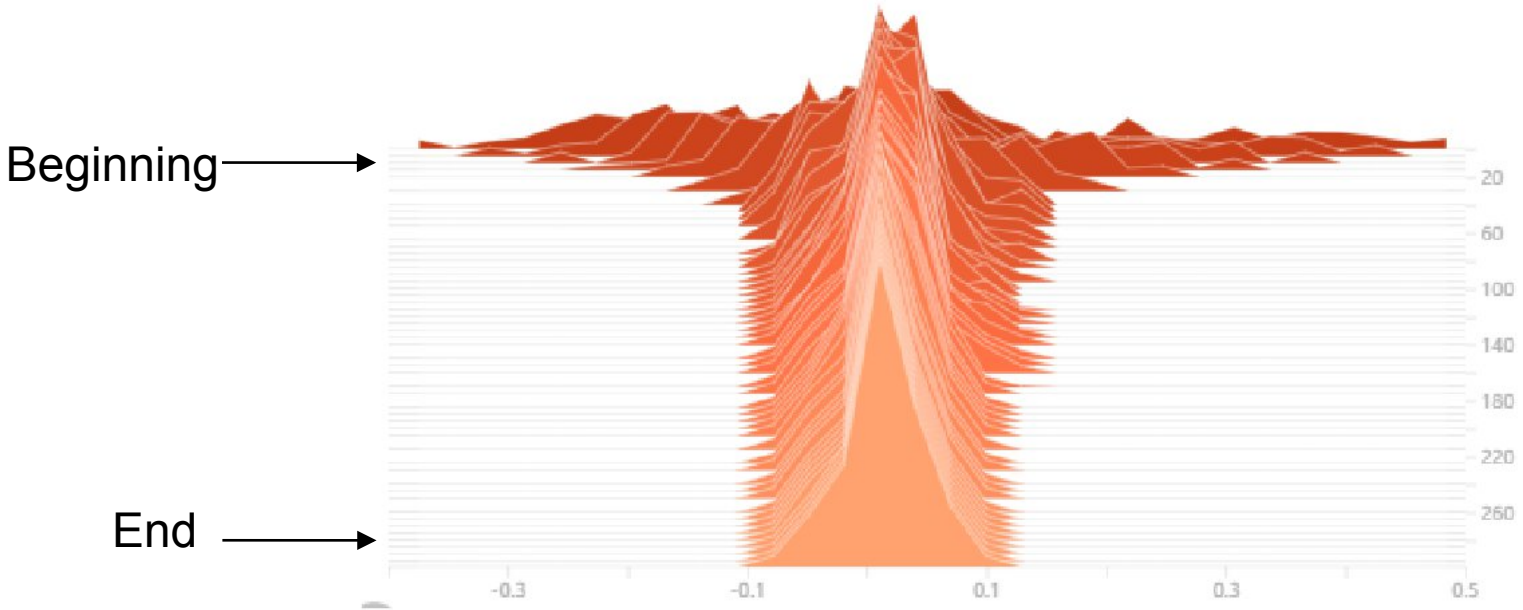


MUON is scalable for llm training, 2025, Jingyuan Liu

In Transformers, Muon can be applied independently to each attention head

Weight decay

Distribution of weights during learning:



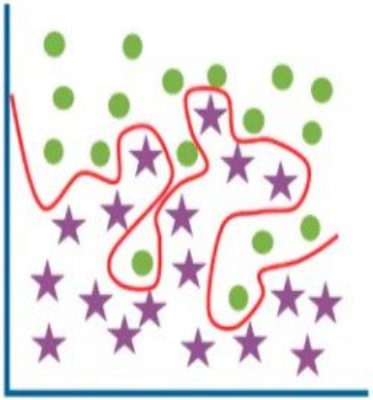
During training the weights tend toward zero

Weight decay

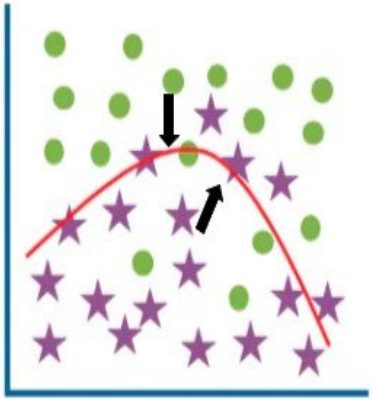
$$\theta_t \leftarrow (1 - \boldsymbol{\lambda})\,\theta_{t-1} - lr \times g_t$$

λ : *weight decay* parameter *(between 0 and 0.1)*

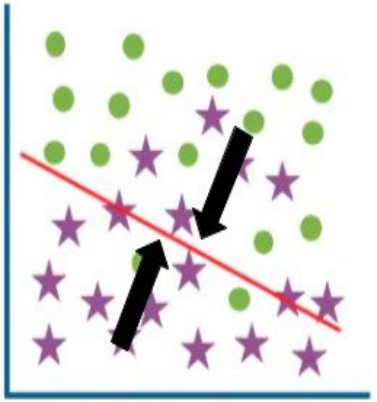
Overfitting
(λ too small or null)



Good generalization
(λ is optimal)



Underfitting
(λ too high)



The weight decay technique force the weights to converge towards values close to zero

Weight decay and decoupled weight decay

Problem : ADAM

For i = 1 to ...

$$g_i = \nabla_\theta f_i(\theta_{i-1}) + \lambda\theta_{i-1}$$

$$m_i = \beta_1 * m_{i-1} + (1 - \beta_1) * g_i$$

$$v_i = \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}$$

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^i}$$

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i$$

Return θ_i

Weight decay

Weight decay is affected by Adam's momentum and adaptive scaling

Solution : ADAMW

For i = 1 to ...

$$g_i = \nabla_\theta f_i(\theta_{i-1})$$

$$m_i = \beta_1 * m_{i-1} + (1 - \beta_1) * g_i$$

$$v_i = \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}$$

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^i}$$

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i - \alpha\lambda\theta_{i-1}$$

Return θ_i

Decoupled weight decay

SGD and SGDW are **roughly equivalent** in performance.
However **AdamW is noticeably better than Adam!!**

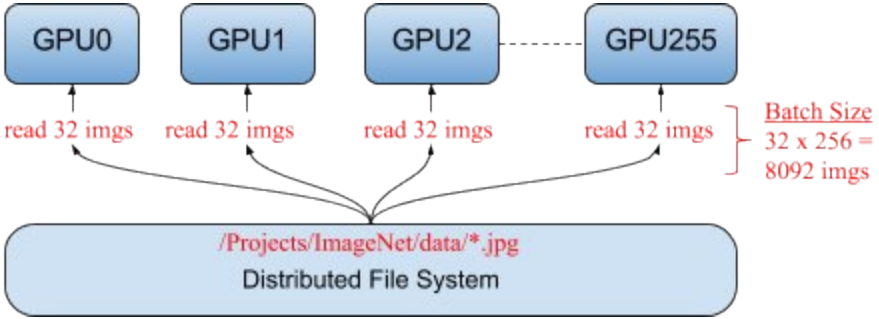
Optimization of large batches

Large batches issues ◂

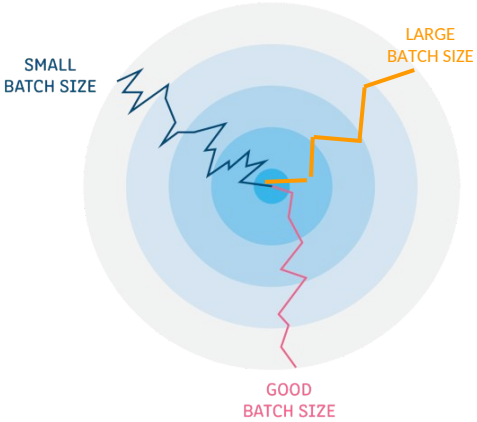
Learning Rate Scaling & Batch Schedulers ◂

Large batches optimizers ◂

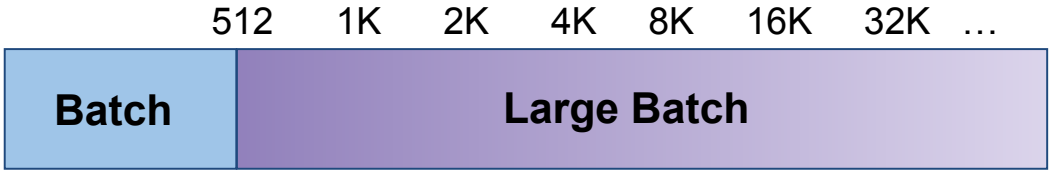
Large Batches with Data Parallelism



Data Parallelism: This parallelism generates very large batches



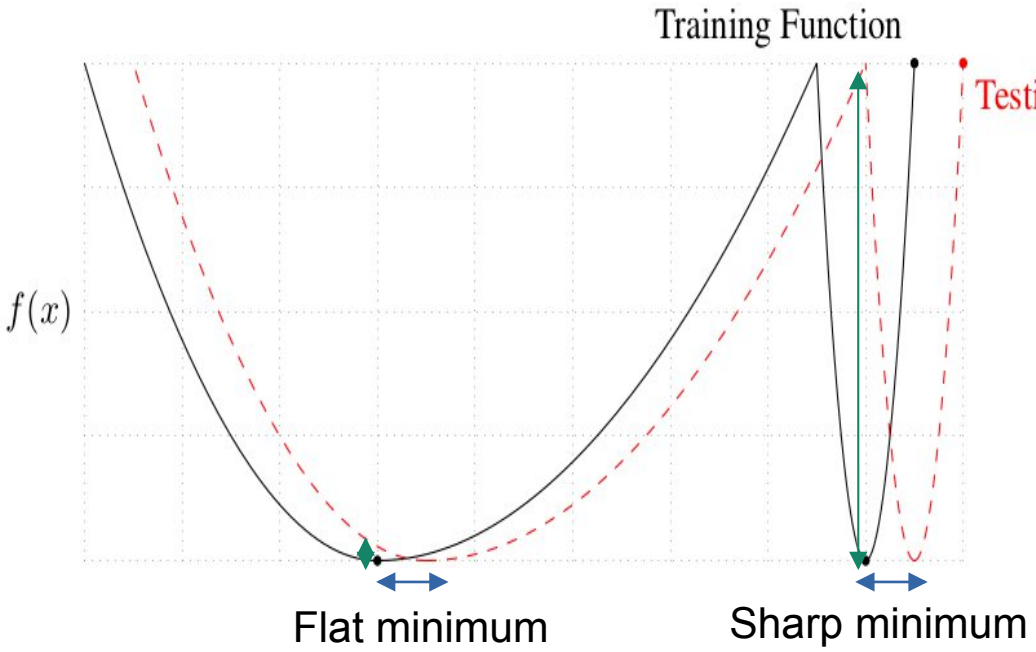
Problem:
Batch that are too large (> 512) tend to result in **poorer performance**

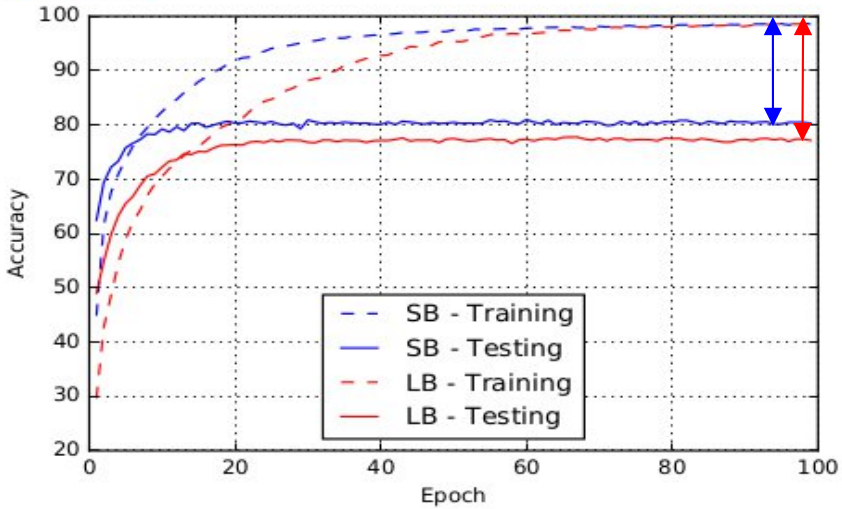


Large Batches

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, Ping Tak Peter Tang





Comparison of training a convolutional network with small batch (SB) and large batch (LB) on CIFAR 10

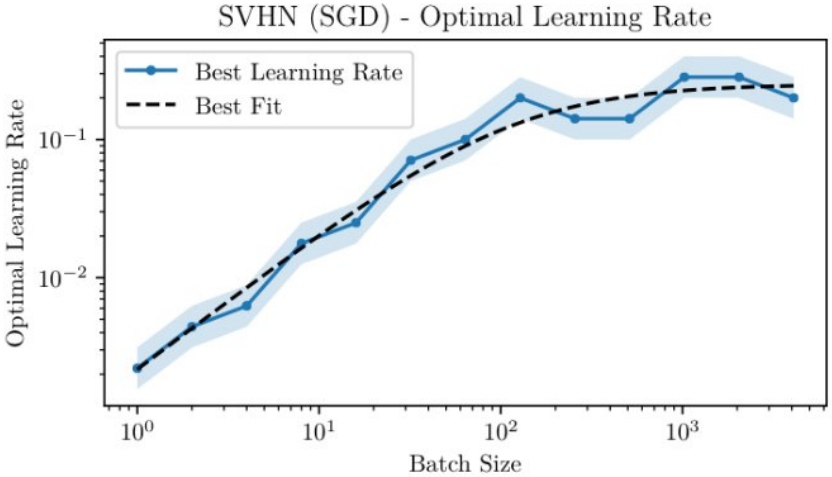
The *larger the batch*, the more the model tends to converge towards **Sharp minima**.

Large Batches : Learning rate scaling

When the *size of the global* batch **is considerably increased**, it is often *necessary* to **scale the learning rate :**

Relationship Between Learning Rate and Batch Size : ADAM $\alpha \rightarrow \sqrt{N} * \alpha$

SGD $\alpha \rightarrow N * \alpha$



An Empirical Model of Large-Batch Training
Sam McCandlish, Jared Kaplan, Dario Amodei

Beyond a certain point, training reaches a plateau, and larger batches provide little additional benefit.

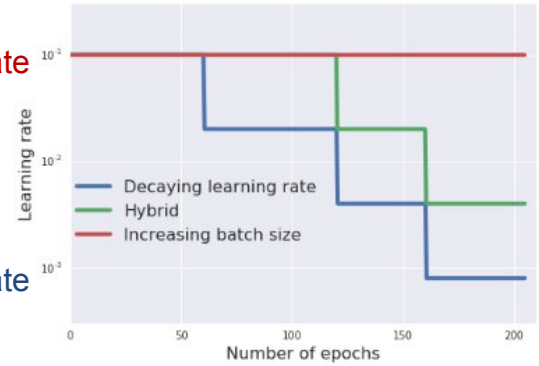
Batch Size Scheduler

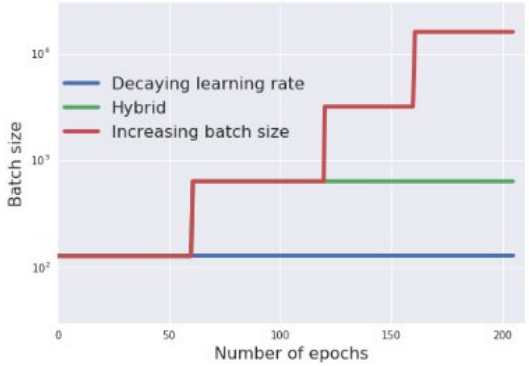
=> Alternative to Learning Rate Scheduler

Don’t decay the learning rate, increase the batch size

High Learning Rate

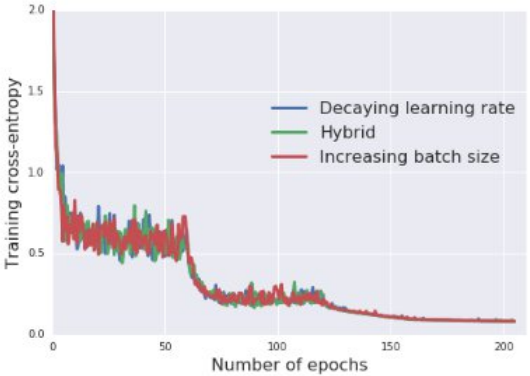
Low Learning Rate

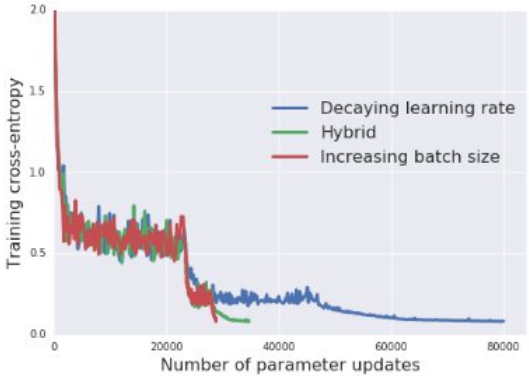




Large Batch

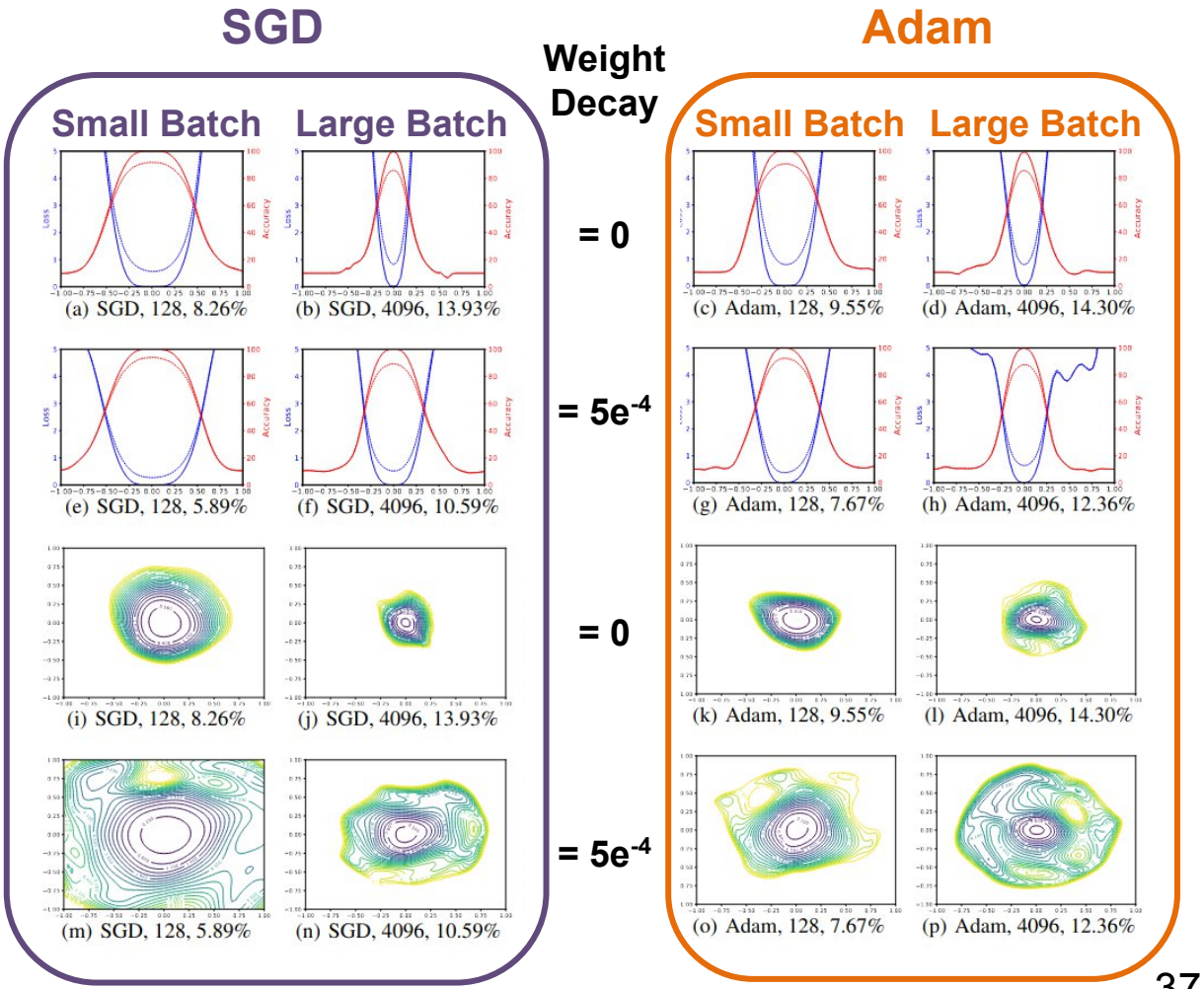
Small Batch





Large Batches

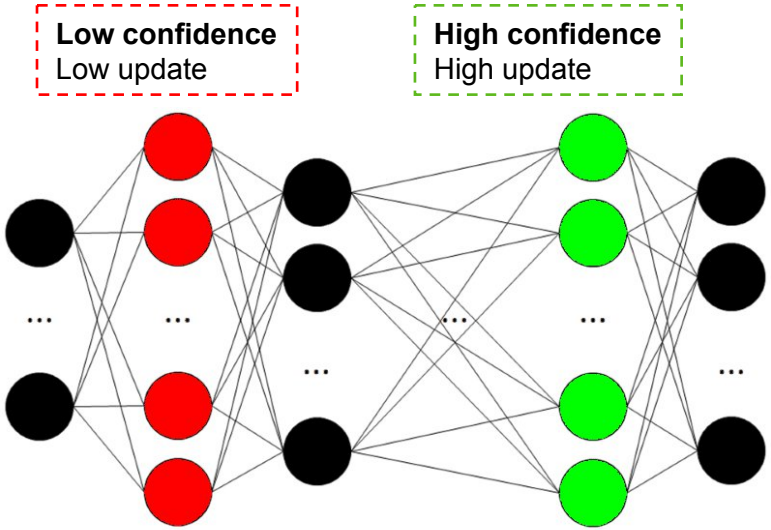
Small Batch	Large Batch
Flat Minimum	Sharp Minimum
- Test Loss	+ Test Loss
Slow Descent	Fast Descent



Layer-wise Adaptive Rate Scaling

Adapt the effective learning rate of each layer using a trust ratio based on the relative norms of its weights and gradients.

$$\lambda^{(l)} = \alpha_{\text{trust}} \frac{\left\| w^{(l)} \right\|}{\left\| g^{(l)} \right\| + \beta \left\| w^{(l)} \right\|} \approx \alpha_{\text{trust}} \frac{\text{size of weights}}{\text{size of proposed update}}$$



Optimizer implementation

SGD

```
import torch.optim as opt

SGD_optimizer = opt.SGD(params, lr, momentum=0, weight_decay=0, nesterov=False, ...)
```

ADAMW

```
import torch.optim as opt

ADAM_optimizer = opt.AdamW(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0.05,..)
```

LAMB
(LARS with adam)

```
from apex.optimizers import FusedLamb

LAMB_optimizer = FusedLamb(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0,
adam_w_mode=True)
```

Muon

```
import torch.optim as opt

MUON_optimizer = opt.Muon(params, lr=0.02, Momentum=0.95, weight_decay=0.05, ...)
```

Are large-batch optimizers still useful?

LARS enables very large-batch CNN training, but the original paper does **not prove it is necessary**.

Source: Large Batch Training of Convolutional Networks

LARS may be better tuned rather than inherently better, **standard optimizers can outperform it.**

Source: A Large Batch Optimizer Reality Check

Muon is becoming a standard choice for training frontier LLMs, especially with large batches.

Source: Muon is Scalable for LLM Training

BLOOM example

95281 steps (116.8 days)

AdamW,

β1=0.9, β2=0.95, eps=1e−8

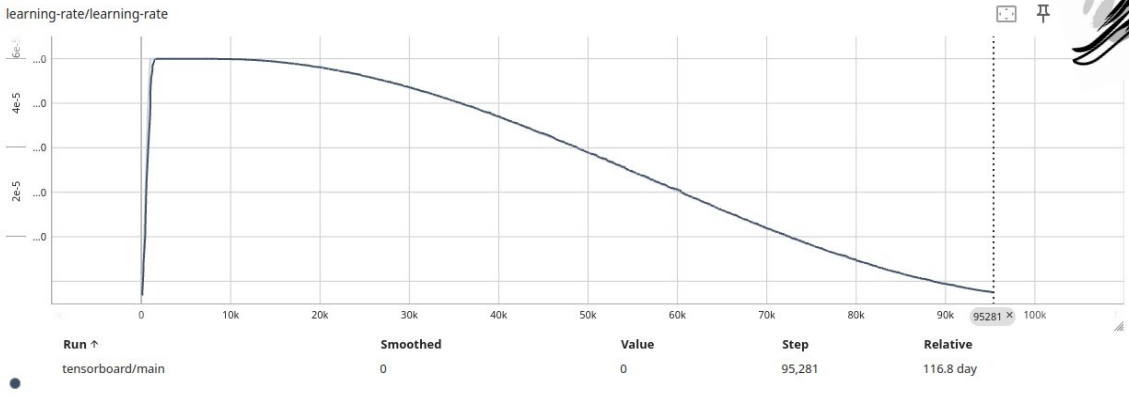
Weight Decay of 0.1

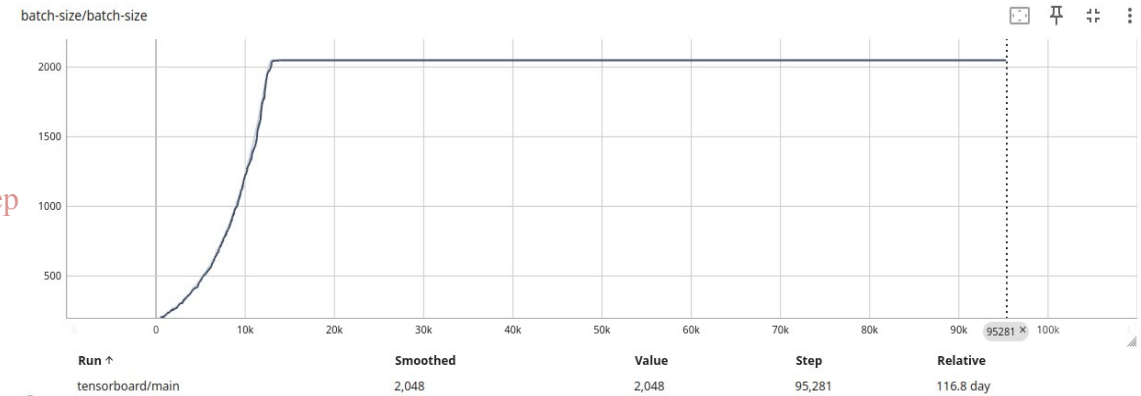
LR Scheduler

- peak=6e-5
- warmup over 375M tokens
- cosine decay for learning rate down to 10% of its value, over 410B tokens

Batch Scheduler

- start from 32k tokens (GBS=16)
- increase linearly to 4.2M tokens/step (GBS=2048) over ~20B tokens
- then continue at 4.2M tokens/step (GBS=2048) for 430B tokens





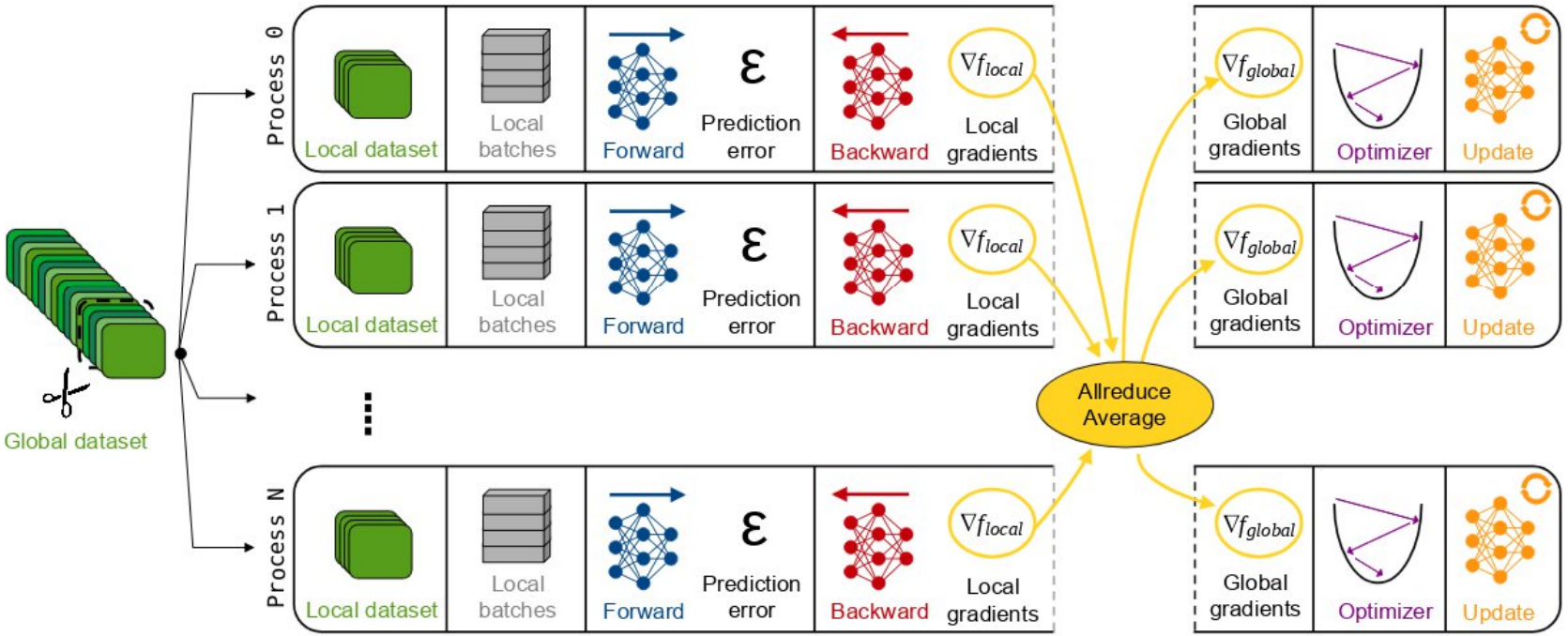
Reducing Optimizer Communication Costs

Communication optimization◂

PowerSGD ◂

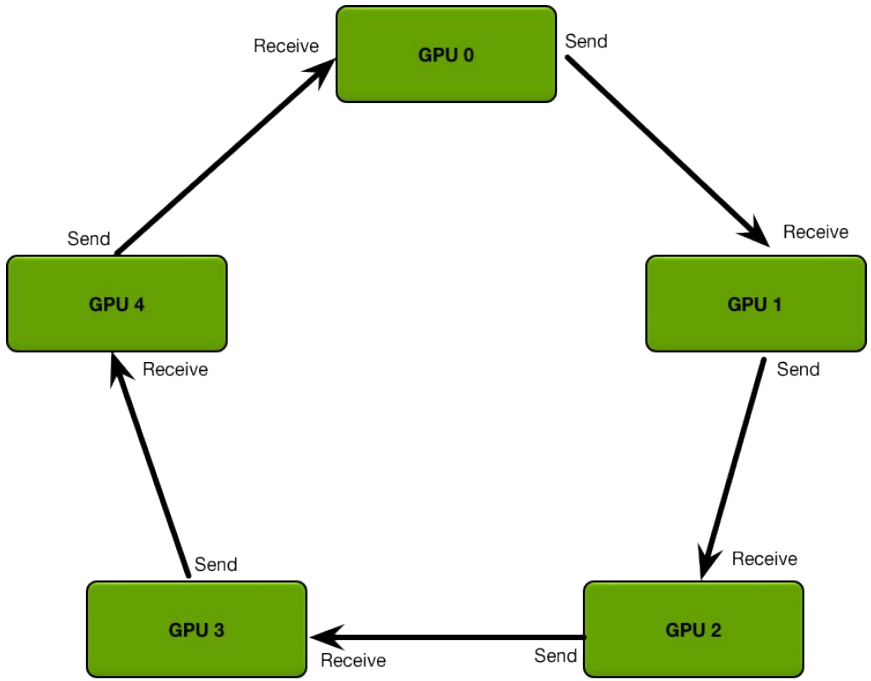
DiLoCo ◂

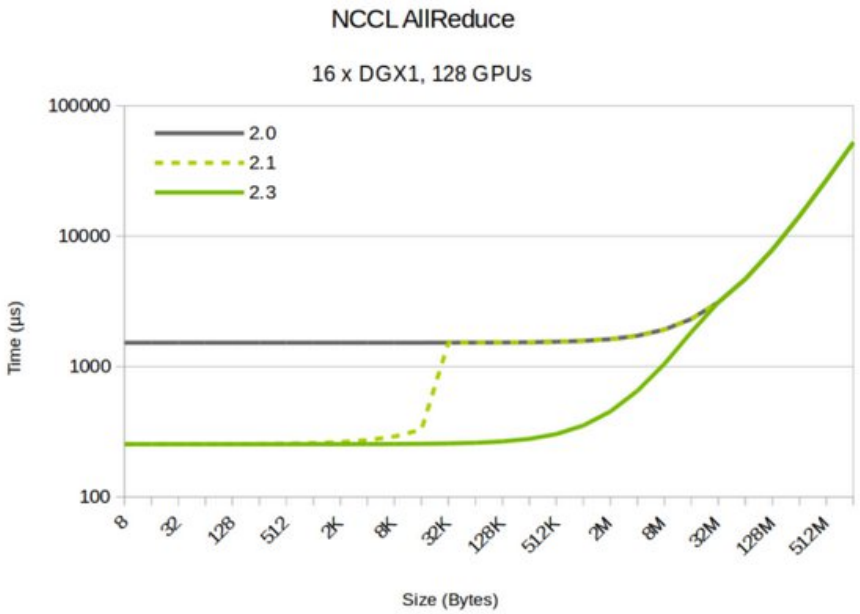
Distributed Data Parallel bottleneck



Communication optimization

Ring all reduce



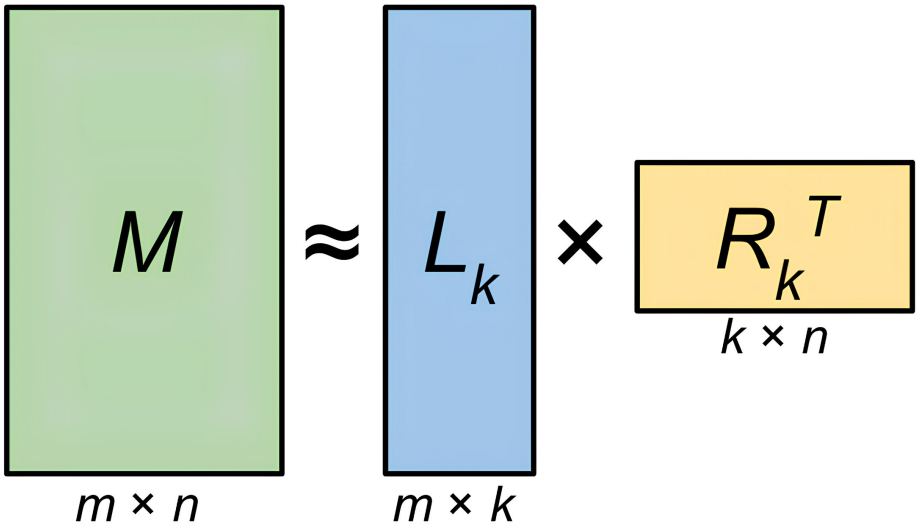


The NCCL version can affect performance

Each GPU sends a fraction of the gradient to the next GPU over N−1 communication steps.

Gradient compression - PowerSGD

The idea is to decompose the weight matrix into two smaller matrices



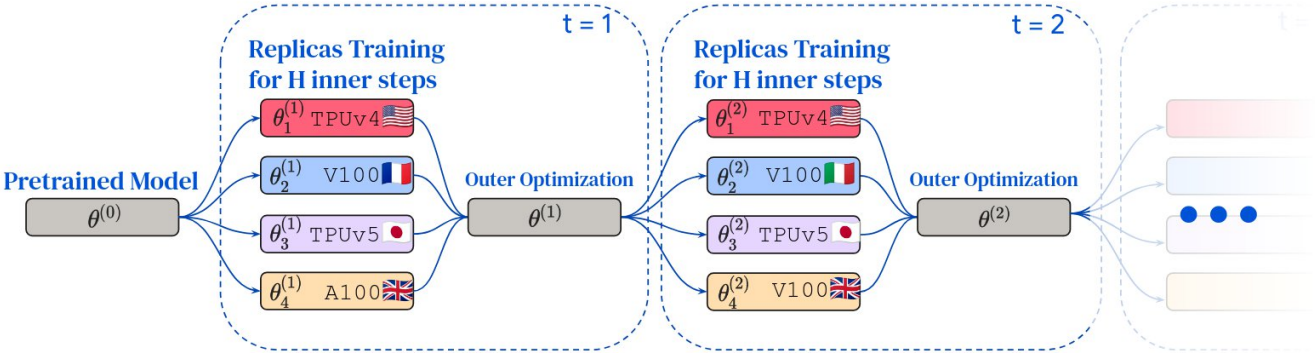
This reduces communication by transmitting two smaller matrices.

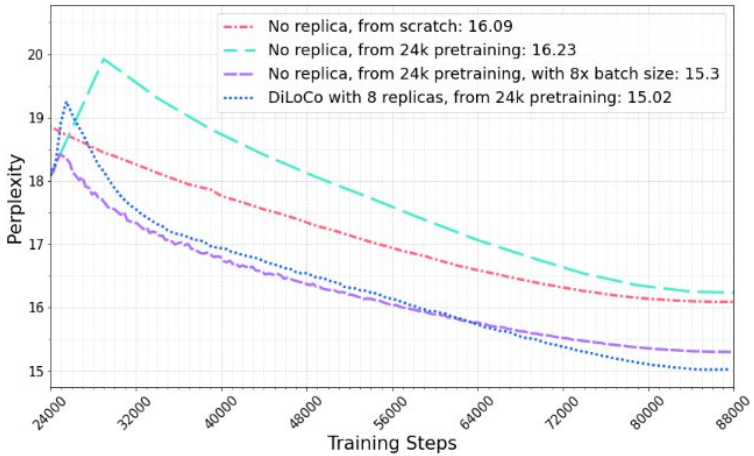
Asynchronous Optimization - DiLoCo

Algorithm 1 DiLoCo Algorithm

Require: Initial model $\theta^{(0)}$
Require: k workers
Require: Data shards $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$
Require: Optimizers `InnerOpt` and `OuterOpt`

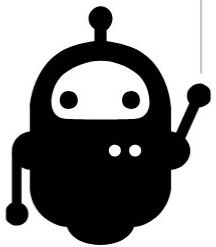
for `outer step` $t = 1 \ldots T$ **do**
  for `worker` $i = 1 \ldots k$ **do**
    $\theta_i^{(t)} \leftarrow \theta^{(t-1)}$
    for `inner step` $h = 1 \ldots H$ **do**
      $x \sim \mathcal{D}_i$
      $\mathcal{L} \leftarrow f(x, \theta_i^{(t)})$
      ▷ Inner optimization:
      $\theta_i^{(t)} \leftarrow \texttt{InnerOpt}(\theta_i^{(t)}, \nabla_{\mathcal{L}})$
    end for
  end for
  ▷ Averaging outer gradients:
  $\Delta^{(t)} \leftarrow \frac{1}{k} \sum_{i=1}^{k} (\theta^{(t-1)} - \theta_i^{(t)})$
  ▷ Outer optimization:
  $\theta^{(t)} \leftarrow \texttt{OuterOpt}(\theta^{(t-1)}, \Delta^{(t)})$
end for





LLM training may not always require globally synchronous gradients at every step

Pratice : Learning rate + Optimiseurs



Goals :

- Edit the **learning rate scheduler**
- Edit the **optimizer**
- Do training with **large batches**

From **JupyterHub**:

- Launch an interactive instance
- Go to the tp_optimizers folder
- Open the DLO-JZ_Optimizers notebook